
Trabajo Práctico 2:

Ajuste y optimización de curvas de Bézier

Métodos Computacionales, 2026

1. Introducción

En el primer trabajo práctico estudiamos las curvas de Bézier como objetos geométricos definidos a partir de puntos de control. Vimos que una curva puede evaluarse mediante combinaciones lineales de funciones base, y que esa representación permite escribir la curva en forma matricial.

En este segundo trabajo práctico vamos a estudiar el problema inverso: en lugar de partir de los puntos de control para construir la curva, vamos a partir de puntos observados y buscar una curva de Bézier que los aproxime lo mejor posible. Hacia el final del trabajo, usaremos las mismas herramientas para diseñar automáticamente una trayectoria suave y segura para una podadora robot. Esto nos permitirá conectar curvas de Bézier con mínimos cuadrados, formas cuadráticas, matrices simétricas, regularización y métodos iterativos de optimización.

El trabajo deberá resolverse en Python, preferentemente en un notebook de Jupyter. En todos los ejercicios se espera que acompañen el código con explicaciones breves, gráficos y una interpretación de los resultados obtenidos.

2. Objetivos

Esperamos que logren:

- formular el ajuste de una curva como un problema de mínimos cuadrados;
- interpretar la función de error como una forma cuadrática;
- reconocer el rol de la matriz simétrica $A^T A$ en el problema de ajuste;
- implementar gradiente descendiente y método de Newton para minimizar una función cuadrática;
- incorporar términos de regularización en una función de costo;
- diseñar y evaluar una trayectoria óptima en una aplicación geométrica simple;
- comparar curvas de distinto grado y elegir el grado mínimo necesario para resolver un problema;
- visualizar y analizar numéricamente los resultados obtenidos.

3. Temario

- | | |
|----------------------------|---------------------------|
| ■ Curvas de Bézier | ■ Matrices simétricas |
| ■ Polinomios de Bernstein | ■ Formas cuadráticas |
| ■ Representación matricial | ■ Regularización |
| ■ Mínimos cuadrados | ■ Gradiente descendiente |
| ■ Ecuaciones normales | ■ Método de Newton |
| ■ Proyecciones | ■ Visualización en Python |

4. Uso de IA generativa

El uso de herramientas de IA generativa está permitido en este trabajo práctico, siempre que sea un soporte para el aprendizaje y no un reemplazo del trabajo propio. Toda asistencia deberá estar documentada en la entrega: qué herramienta se usó y para qué se usó.

Categoría	Descripción	Estado
Hacer todo el trabajo por mí	Usar IA para completar el TP porque no tenía tiempo, estaba cansado o no le veía sentido al trabajo.	PROHIBIDO. Hablá con el docente para una alternativa.
Hacer el trabajo repetitivo	Usar IA para completar partes del TP que considero repetitivas o que no aportan aprendizaje.	DESALENTADO. Consultá con el docente antes.
Arrancar / destrabarme	Usar IA para resumir consignas, planificar, armar un esquema o destrabarme cuando estoy trabado.	PERMITIDO. Documentá su uso en la entrega.
Pedir feedback	Usar IA para revisar mi razonamiento, detectar errores o recibir sugerencias sobre mi trabajo.	PERMITIDO. Documentá su uso en la entrega.
Ayudarme a aprender	Usar IA para explicar conceptos nuevos o difíciles paso a paso, con ejemplos o analogías.	FOMENTADO. Mencioná su uso en la entrega.
Amplificar mi trabajo	Usar IA para ir más allá de lo que pide la consigna, explorar extensiones o profundizar.	FOMENTADO. Documentá su uso en la entrega.

5. Condiciones de entrega y modalidad de aprobación

La entrega deberá realizarse en un único archivo comprimido en formato .zip. El archivo deberá contener un notebook de Jupyter ya ejecutado, con la resolución completa del trabajo práctico. El notebook debe incluir el código, las explicaciones, los gráficos solicitados y las respuestas conceptuales. También se deberá entregar una presentación corta en formato .pdf que resuma/explice brevemente las soluciones principales del trabajo práctico. Esta presentación podrá ser utilizada como material de apoyo durante la instancia oral, en caso de que sea necesario. Además, deberán incluir un archivo uso_ia.txt donde indiquen si usaron o no herramientas de IA generativa. En caso de haberlas usado, deberán describir brevemente en qué ejercicios y con qué propósito. La estructura esperada de la entrega es:

```
apellido-1_nombre-1_apellido-2_nombre-2_tp2.zip
|
|-- tp2.ipynb
|-- uso_ia.txt
|-- presentacion.pdf
|-- figuras/
|   |-- ejercicio_XX.png
|   |-- ejercicio_YY.png
|   |-- ...
```

Entregas que no respeten los nombres y anidamiento de los archivos indicados arriba se considerarán desaprobadas.

El notebook debe poder ejecutarse de principio a fin sin errores y debe ser autocontenido (no utilizar archivos auxiliares). Se recomienda usar solamente numpy y matplotlib. Si utilizan otra biblioteca, deberán justificarlo brevemente. **La entrega es por campus (sin excepciones) y hay tiempo hasta el domingo 21/6 a las 23:59.** Las entregas tardías se considerarán desaprobadas.

La evaluación consistirá únicamente en una instancia oral en la que deberán responder preguntas sobre los ejercicios entregados. Las preguntas podrán estar relacionadas, por ejemplo, con la implementación de las funciones, las decisiones tomadas en el código, la interpretación de los gráficos o la formulación teórica que respalda alguna respuesta. **Los alumnos de la Sección 1 rendirán el oral el Lunes 22/6 en el horario de teórica. Los alumnos de la Sección 2 rendirán el oral el Martes 23/6 en el horario de práctica.**

Importante: Lo que efectivamente determinará la aprobación del trabajo práctico será el desempeño en la instancia oral. De nada sirve presentar una entrega correcta o incluso perfecta si luego no se puede explicar cómo fue resuelta, qué decisiones se tomaron, qué conceptos están involucrados o por qué los resultados obtenidos tienen sentido. El uso de herramientas de IA generativa está permitido, pero no los exime de comprender y poder explicar su propia entrega. Esperamos que sean muy responsables con esto. **Bajo ningún punto de vista se aceptará como respuesta que una solución fue generada por una herramienta de IA y que no se sabe cómo funciona.**

Ejercicios

En cada ejercicio se indica un tag de uso de IA generativa. La etiqueta **Sin IA** significa que el ejercicio debe resolverse sin asistencia de IA. La etiqueta **Asistencia para destrabar/corregir** significa que pueden usar IA para destrabarse, revisar errores o pedir feedback, pero la solución debe ser propia y verificada. La etiqueta **IA necesaria** significa que se espera que usen IA como herramienta de asistencia para programar, debugging o explorar alternativas, pero comprendiendo cabalmente cada uno de los aspectos de la solución, y siendo capaces de replicarla por sus propios medios en caso de ser necesario.

1. **Sin IA** Funciones base de Bernstein

Considerar las funciones base de Bernstein de grado 3:

$$b_0(t) = (1-t)^3, \quad b_1(t) = 3(1-t)^2t, \quad b_2(t) = 3(1-t)t^2, \quad b_3(t) = t^3.$$

- Implementar en Python una función `bernstein3(t)` que reciba un arreglo de valores de t y devuelva una matriz con las cuatro funciones evaluadas en esos valores.
- Graficar las cuatro funciones en el intervalo $[0, 1]$.
- Verificar numéricamente que, para todo $t \in [0, 1]$, se cumple

$$b_0(t) + b_1(t) + b_2(t) + b_3(t) = 1.$$

- Asistencia para destrabar/corregir** Explicar brevemente por qué esta propiedad es importante para interpretar una curva de Bézier como combinación de puntos de control.

2. **Sin IA** Evaluación matricial de una curva de Bézier

Sea una curva de Bézier cúbica con puntos de control

$$P_0 = (0, 0), \quad P_1 = (1, 3), \quad P_2 = (4, 3), \quad P_3 = (5, 0).$$

- Construir una matriz $A \in \mathbb{R}^{m \times 4}$ evaluando las funciones de Bernstein en $m = 100$ valores equiespaciados de $t \in [0, 1]$.
- Construir una matriz $P \in \mathbb{R}^{4 \times 2}$ cuyas filas sean los puntos de control.

- c) Calcular los puntos de la curva mediante el producto

$$C = AP.$$

- d) Graficar la curva, los puntos de control y el polígono de control.
e) Indicar las dimensiones de A , P y C , y explicar qué representa cada matriz.

3. **Sin IA** Generación de datos con ruido

A partir de la curva del ejercicio anterior, generar un conjunto de datos observados.

- a) Tomar $m = 30$ valores equiespaciados $t_1, \dots, t_m \in [0, 1]$.
b) Evaluar la curva exacta en esos valores.
c) Agregar ruido gaussiano pequeño a cada coordenada. Por ejemplo:

$$q_i = B(t_i) + \eta_i,$$

donde $\eta_i \in \mathbb{R}^2$ es un vector aleatorio con media cero. Fijar una semilla aleatoria usando `np.random.seed` para que el resultado sea reproducible.

- d) Graficar la curva original y los puntos ruidosos obtenidos.

4. **Sin IA** Ajuste de una curva de Bézier por mínimos cuadrados

En este ejercicio deberán ajustar una curva de Bézier cúbica a partir de los puntos ruidosos del ejercicio anterior.

- a) Construir la matriz $A \in \mathbb{R}^{m \times 4}$ asociada a los valores $t_1, \dots, t_m$.
b) Sea $Q \in \mathbb{R}^{m \times 2}$ la matriz que contiene los puntos observados. Resolver el problema

$$\hat{P} = \arg \min_{P \in \mathbb{R}^{4 \times 2}} \|AP - Q\|^2.$$

- c) Calcular $\hat{P}$ usando la fórmula de ecuaciones normales:

$$\hat{P} = (A^T A)^{-1} A^T Q.$$

- d) Graficar en una misma figura los puntos observados, la curva original y la curva ajustada.
e) Comparar los puntos de control originales con los puntos de control estimados. ¿Son exactamente iguales? ¿Por qué?
f) Repetir el procedimiento anterior para distintos niveles de ruido. Por ejemplo, usar

$$\sigma \in \{0,00, 0,02, 0,05, 0,10, 0,20\},$$

donde σ controla la desviación estándar del ruido agregado a los puntos de la curva.

- g) Para cada nivel de ruido, ajustar una nueva curva de Bézier por mínimos cuadrados y graficar en una misma figura la curva original junto con las curvas ajustadas obtenidas.

- h) Calcular, para cada valor de σ , el error cuadrático medio entre la curva original y la curva ajustada:

$$\text{ECM} = \frac{1}{m} \sum_{i=1}^m \|B(t_i) - \hat{B}(t_i)\|^2.$$

Construir una tabla con los valores de σ y el error obtenido.

- i) Describir brevemente qué ocurre cuando aumenta el nivel de ruido. ¿La curva ajustada sigue recuperando razonablemente la forma original? ¿Qué cambia en los puntos de control estimados?

5. **Sin IA** La función de error como forma cuadrática

En el ejercicio anterior resolvimos el ajuste de una curva de Bézier minimizando un error de mínimos cuadrados. Para estudiar la forma algebraica de esa función de error, analizaremos primero el caso de una sola coordenada. Esto equivale a mirar sólo las coordenadas x o sólo las coordenadas y de los puntos observados.

Considerar entonces la función

$$E(p) = \|Ap - q\|^2,$$

con $A \in \mathbb{R}^{m \times 4}$, $p \in \mathbb{R}^4$ y $q \in \mathbb{R}^m$.

- a) Expandir algebraicamente $E(p)$ y mostrar que puede escribirse como

$$E(p) = p^T A^T A p - 2q^T A p + q^T q.$$

- b) Identificar la parte cuadrática de la expresión.
c) Verificar numéricamente que $A^T A$ es una matriz simétrica.
d) Calcular los valores propios de $A^T A$ usando `np.linalg.eig` o `np.linalg.eigh`.
e) ¿Todos los valores propios son positivos? ¿Qué sugiere esto sobre la forma de la función de error?

6. **Sin IA** Ecuaciones normales a partir del gradiente

A partir de

$$E(p) = \|Ap - q\|^2,$$

el gradiente está dado por

$$\nabla E(p) = 2A^T(Ap - q).$$

- a) Implementar una función `gradiente(p, A, q)` que calcule $\nabla E(p)$.
b) Evaluar el gradiente en un vector inicial cualquiera, por ejemplo $p = (0, 0, 0, 0)^T$.
c) Evaluar el gradiente en la solución de mínimos cuadrados $\hat{p}$.
d) Verificar que el gradiente en $\hat{p}$ es aproximadamente cero.
e) Mostrar que imponer $\nabla E(p) = 0$ conduce a las ecuaciones normales:

$$A^T A p = A^T q.$$

7. **Asistencia para destrabar/corregir** **Gradiente descendiente para ajustar una curva**

Implementar gradiente descendiente para minimizar $E(p) = \|Ap - q\|^2$.

a) Inicializar p_0 de manera aleatoria.

b) Implementar la actualización

$$p_{t+1} = p_t - \alpha 2A^T(Ap_t - q).$$

c) Ejecutar el método durante una cantidad fija de iteraciones.

d) Guardar el valor de $E(p_t)$ en cada iteración y graficarlo.

e) Comparar el resultado final con la solución obtenida por ecuaciones normales y graficar.

8. **Asistencia para destrabar/corregir** **Efecto de la tasa de aprendizaje**

Usar el algoritmo del ejercicio anterior para estudiar el efecto de la tasa de aprendizaje α .

a) Probar al menos cuatro valores distintos de α , procurando que difieran entre sí en uno o más órdenes de magnitud. Por ejemplo, pueden probar valores como 10^{-4} , 10^{-3} , 10^{-2} y 10^{-1} , o ajustar esa escala si el método diverge demasiado rápido.

b) Para cada valor, graficar el error en función de la iteración.

c) Identificar un caso en el que el método converge lentamente.

d) Identificar, si es posible, un caso en el que el método diverge.

e) Explicar en palabras qué rol cumple α en el método.

9. **Asistencia para destrabar/corregir** **Método de Newton para el mismo problema**

Para la función cuadrática

$$E(p) = \|Ap - q\|^2,$$

el Hessiano es

$$H = 2A^T A.$$

a) Implementar el método de Newton usando la actualización

$$p_{t+1} = p_t - H^{-1} \nabla E(p_t).$$

b) Inicializar el método desde un vector aleatorio p_0 .

c) Comparar p_1 con la solución de mínimos cuadrados.

d) Explicar por qué Newton llega al mínimo en una sola iteración para una función cuadrática.

e) Repetir el experimento para las coordenadas x e y , y graficar la curva resultante.

10. **Asistencia para destrabar/corregir** Ajuste con extremos fijos

En muchas aplicaciones geométricas se desea ajustar una curva a datos, pero manteniendo fijos los extremos. En este ejercicio deberán imponer

$$P_0 = q_1, \quad P_3 = q_m.$$

a) Escribir la curva como

$$B(t) = b_0(t)P_0 + b_1(t)P_1 + b_2(t)P_2 + b_3(t)P_3.$$

b) Pasar los términos conocidos al lado derecho para obtener un problema de mínimos cuadrados sólo en P_1 y P_2 .

c) Construir la nueva matriz $A_{\text{int}} \in \mathbb{R}^{m \times 2}$ usando sólo las columnas asociadas a $b_1(t)$ y $b_2(t)$.

d) Resolver el problema para estimar P_1 y P_2 .

e) Graficar y comparar el ajuste con extremos fijos contra el ajuste sin restricciones.

11. **IA necesaria** Aplicación: una trayectoria para la podadora robot

Una podadora robot debe desplazarse desde un punto de origen $O = (0, 0)$ hasta un punto destino $D = (12, 0)$ dentro de un jardín. El jardín tiene paredes y varios arbustos circulares que la podadora debe evitar.

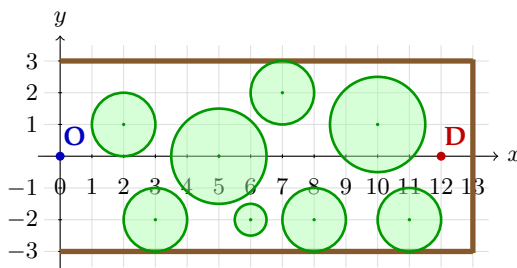
A diferencia de los ejercicios anteriores, ahora no fijaremos de antemano el grado de la curva. La trayectoria será representada mediante una curva de Bézier de grado n :

$$B_n(t) = \sum_{k=0}^n b_{k,n}(t)P_k, \quad b_{k,n}(t) = \binom{n}{k}(1-t)^{n-k}t^k, \quad t \in [0, 1].$$

Los extremos estarán siempre fijos:

$$P_0 = O = (0, 0), \quad P_n = D = (12, 0),$$

y los puntos intermedios $P_1, \dots, P_{n-1}$ serán las variables a optimizar. El objetivo será encontrar automáticamente una trayectoria factible, empezando con grados bajos y aumentando el grado sólo cuando sea necesario.



Los obstáculos del jardín son los siguientes:

j	c_j	r_j
1	(2, 1)	1
2	(3, -2)	1
3	(5, 0)	1,5
4	(7, 2)	1
5	(8, -2)	1
6	(6, -2)	0,5
7	(10, 1)	1,5
8	(11, -2)	1

- a) Implementar en Python una función `bernstein(n, t)` que devuelva la matriz de funciones base de Bernstein de grado n evaluadas en los valores de t recibidos.
- b) Implementar una función `bezier(P, t)` que evalúe una curva de Bézier de grado arbitrario a partir de una matriz de puntos de control $P \in \mathbb{R}^{(n+1) \times 2}$.

- c) Implementar una función que, dados los puntos de una trayectoria muestreada, calcule la distancia mínima a cada obstáculo y determine si hay colisión. Diremos que hay colisión con el obstáculo j si

$$\min_i \|B_n(t_i) - c_j\| < r_j.$$

- d) Usar $N = 200$ valores equiespaciados de $t \in [0, 1]$ para evaluar las trayectorias.
- e) Para cada grado $n \in \{2, 3, 4, 5, 6\}$, construir una trayectoria inicial usando puntos de control equiespaciados entre O y D , pero con una pequeña perturbación vertical. Por ejemplo, pueden usar

$$P_k^{(0)} = \left(12 \frac{k}{n}, a \sin \left(\pi \frac{k}{n} \right) \right), \quad k = 0, \dots, n,$$

con algún valor de a elegido por ustedes.

- f) Graficar las trayectorias iniciales para distintos grados y verificar cuáles colisionan con obstáculos.
- g) Explicar por qué una curva de grado muy bajo puede no tener suficiente flexibilidad para atravesar el jardín sin colisionar, mientras que una curva de grado alto puede producir trayectorias innecesariamente complejas.

12. IA necesaria Optimización automática del grado y de la trayectoria

Queremos encontrar automáticamente una trayectoria que sea corta, suave y que evite los obstáculos. Para eso optimizaremos los puntos de control intermedios de una curva de Bézier de grado n y repetiremos el procedimiento para grados crecientes hasta encontrar una trayectoria factible.

Para cada grado n , definimos el vector de variables

$$\theta = [P_{1x} \ P_{1y} \ P_{2x} \ P_{2y} \ \dots \ P_{n-1,x} \ P_{n-1,y}]^T \in \mathbb{R}^{2(n-1)}.$$

Los puntos P_0 y P_n permanecen fijos.

La función de costo tendrá tres términos:

$$E(\theta) = E_{\text{largo}}(\theta) + \lambda_{\text{suavidad}} E_{\text{suavidad}}(\theta) + \lambda_{\text{obs}} E_{\text{obstáculos}}(\theta).$$

El primer término penaliza trayectorias largas:

$$E_{\text{largo}}(\theta) = \sum_{i=1}^{N-1} \|B_n(t_{i+1}) - B_n(t_i)\|^2.$$

El segundo término penaliza cambios bruscos en los puntos de control mediante diferencias de segundo orden:

$$E_{\text{suavidad}}(\theta) = \sum_{k=1}^{n-1} \|P_{k-1} - 2P_k + P_{k+1}\|^2.$$

El tercer término penaliza la cercanía a los obstáculos:

$$E_{\text{obstáculos}}(\theta) = \sum_{i=1}^N \sum_{j=1}^M [\max(0, r_j + \delta - \|B_n(t_i) - c_j\|)]^2.$$

El parámetro $\delta > 0$ representa una distancia de seguridad adicional alrededor de cada arbusto.

- Implementar en Python una función que convierta un vector θ en la matriz completa de puntos de control P , agregando los extremos fijos P_0 y P_n .
- Implementar los tres términos de la función de costo para un grado arbitrario n . Explicar brevemente la intuición detrás de cada implementación. Cuando sea posible, dar ejemplos de minimización/maximización de cada término.
- Para cada grado $n \in \{2, 3, 4, 5, 6\}$, optimizar la trayectoria usando como punto inicial la curva construida en el ejercicio anterior.

Usar, por ejemplo, los siguientes parámetros:

$$\lambda_{\text{suavidad}} = 0.1, \quad \lambda_{\text{obs}} = 10, \quad \delta = 0.3.$$

Como el gradiente de $E(\theta)$ puede ser difícil de obtener analíticamente, aproximarlos mediante diferencias finitas centrales:

$$\frac{\partial E}{\partial \theta_k} \approx \frac{E(\theta + \varepsilon e_k) - E(\theta - \varepsilon e_k)}{2\varepsilon},$$

donde e_k es el vector canónico asociado a la coordenada θ_k .

Luego, implementar gradiente descendiente usando la actualización

$$\theta_{t+1} = \theta_t - \alpha \nabla E(\theta_t).$$

- Para cada grado, graficar el valor de $E(\theta)$ en función de la iteración.
- Para cada grado, graficar la trayectoria inicial y la trayectoria optimizada sobre el mapa del jardín.
- Definir un criterio de trayectoria factible. Por ejemplo: que no haya colisiones y que la distancia mínima a cualquier obstáculo sea al menos δ .
- Determinar cuál es el menor grado n para el cual el procedimiento encuentra una trayectoria factible. Justificar la elección.

13. Asistencia para destrabar/corregir **Análisis de la trayectoria obtenida**

Una vez encontrada una trayectoria factible, analizaremos cómo cambian los resultados al modificar el grado de la curva y los pesos de la función de costo.

- a) Construir una tabla comparando, para cada grado probado, la longitud aproximada de la trayectoria, la cantidad de colisiones, la distancia mínima a un obstáculo y el valor final de la función de costo.
- b) Comparar visualmente las trayectorias optimizadas para los distintos grados. ¿Qué cambia cuando aumenta el grado?
- c) Repetir el experimento para al menos tres valores distintos de λ_{obs} , por ejemplo:

$$\lambda_{\text{obs}} \in \{1; 10; 100\}.$$

- d) Repetir el experimento para al menos tres valores distintos de $\lambda_{\text{suavidad}}$, por ejemplo:

$$\lambda_{\text{suavidad}} \in \{0; 0,1; 1\}.$$

- e) Elegir una trayectoria final para la podadora robot. La elección no tiene que ser necesariamente la de menor costo numérico: debe estar justificada en términos de seguridad, longitud, suavidad y simplicidad de la curva.
- f) Explicar en un párrafo cómo aparecen en este problema las siguientes ideas de la materia: mínimos cuadrados, formas cuadráticas, regularización, gradiente descendiente y optimización numérica.